FIAP - Engenharia de Software

Global Solution 2026 - CitizenTech

Domain Driven Design - Java

SafeRoute

API REST, persistência Oracle e publicação no Render

Equipe SafeRoute

Bruno Gama Masseiras - RM: 562526

Kauai Rosa de Assis Rocha - RM: 563256

Leandro Afonso Silva Santos Junior - RM: 561344

Lucas Gilmar Pedro da Silva - RM: 566231

Murilo da Silva Basto - RM: 565445

São Paulo

2026

Sumário

1. Proposta, justificativa e objetivos 3

2. Arquitetura e classes de aplicação 4

3. Diagrama de classes: domínio 5

4. Modelo relacional 6

5. Interface: consulta de trajetos 7

6. Interface: operação e ocorrências 8

7. Endpoints e requisições HTTP 9

8. Como executar e acessar 10

9. Verificação e limites 11

1. Proposta, justificativa e objetivos

Durante chuvas intensas, um caminho curto pode ficar impraticável por alagamento ou bloqueio. A SafeRoute centraliza relatos e condições dos trechos para demonstrar como uma aplicação pode apresentar alternativas de circulação, incluindo restrições de acessibilidade.

O território de referência é o Jardim Pantanal, no distrito Jardim Helena, em São Paulo (SP). A Prefeitura registra inundações recorrentes nessa região da várzea do Tietê. A malha real contém 1.188 pontos e 1.427 segmentos extraídos do OpenStreetMap. A previsão regional vem da API Open-Meteo; ela não é tratada como confirmação de alagamento. A validação com moradores permanece como etapa futura.

1.1 Objetivo e público

Implementar uma capacidade central da CitizenTech em Java: cadastrar a rede e seus sensores, registrar ocorrências, apresentar alertas e calcular uma alternativa que respeite os bloqueios conhecidos. O público proposto reúne moradores, trabalhadores e pessoas com mobilidade reduzida.

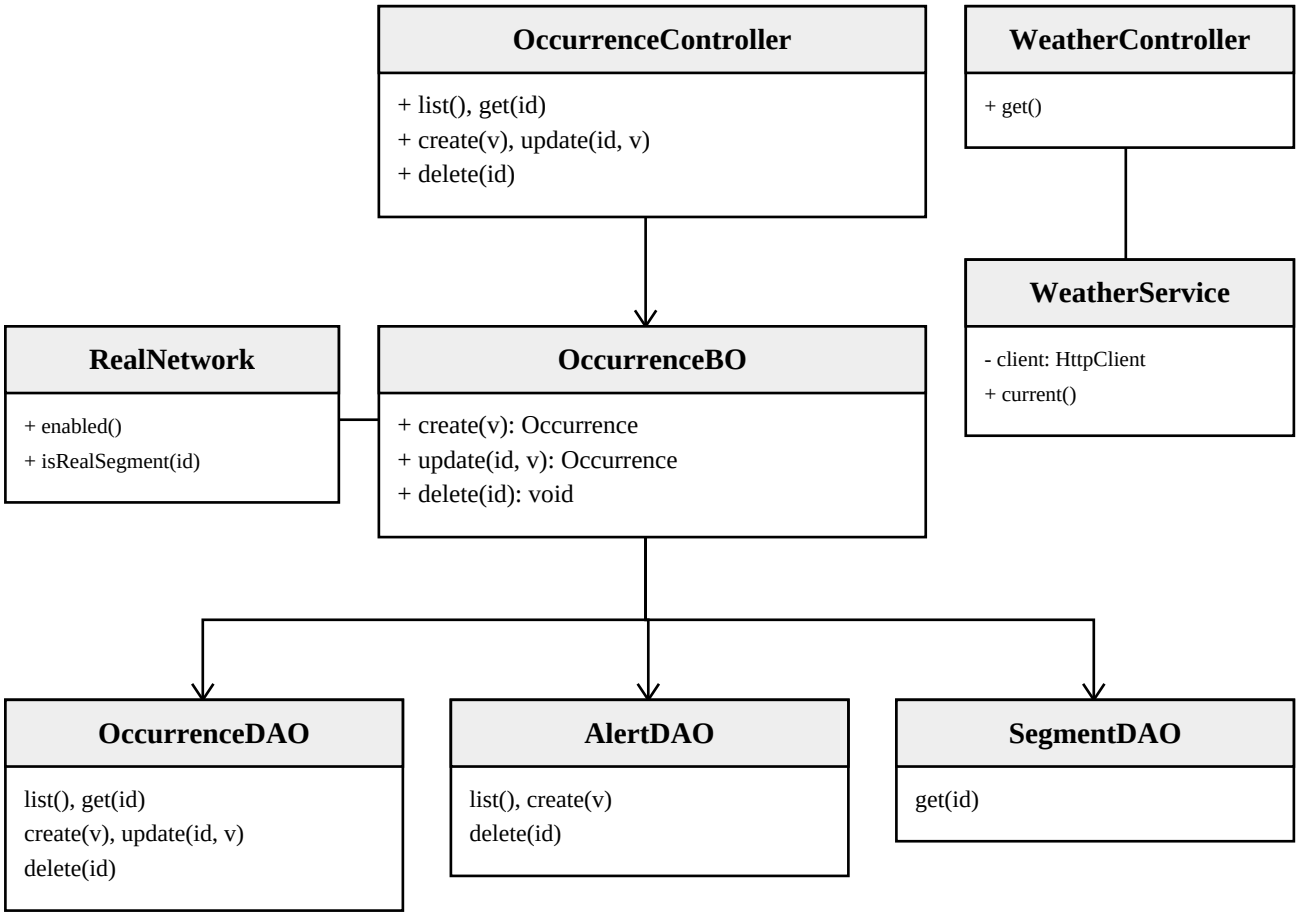
Funcionalidade	Comportamento implementado
Cadastro da rede	CRUD de regiões, pontos, trechos e sensores.
Monitoramento	Previsão Open-Meteo e ocorrências de operadores; testes de sensores separados.
Consulta de rotas	Dijkstra em rede bidirecional, com restrições e custo ponderado.
Alertas	Consulta de ocorrências vigentes na interface web.
Integração	Oracle FIAP, extrato OSM georreferenciado, Leaflet e API Open-Meteo.

1.2 Justificativa e contribuição proposta

A combinação de ocorrências e acessibilidade permite avaliar alternativas antes de priorizar distância. A proposta se relaciona aos ODS 11, 10 e 13. O impacto poderá ser avaliado pela proporção de trajetos que evitam bloqueios e pelo tempo para localizar uma alternativa; não houve medição de impacto real.

2. Arquitetura e classes de aplicação

Classes do pacote com.tlean07.saferoute. Controllers recebem HTTP; BOs aplicam regras; DAOs acessam Oracle. RealNetwork importa e filtra a malha OSM. WeatherService consulta a previsão com cache e timeout.

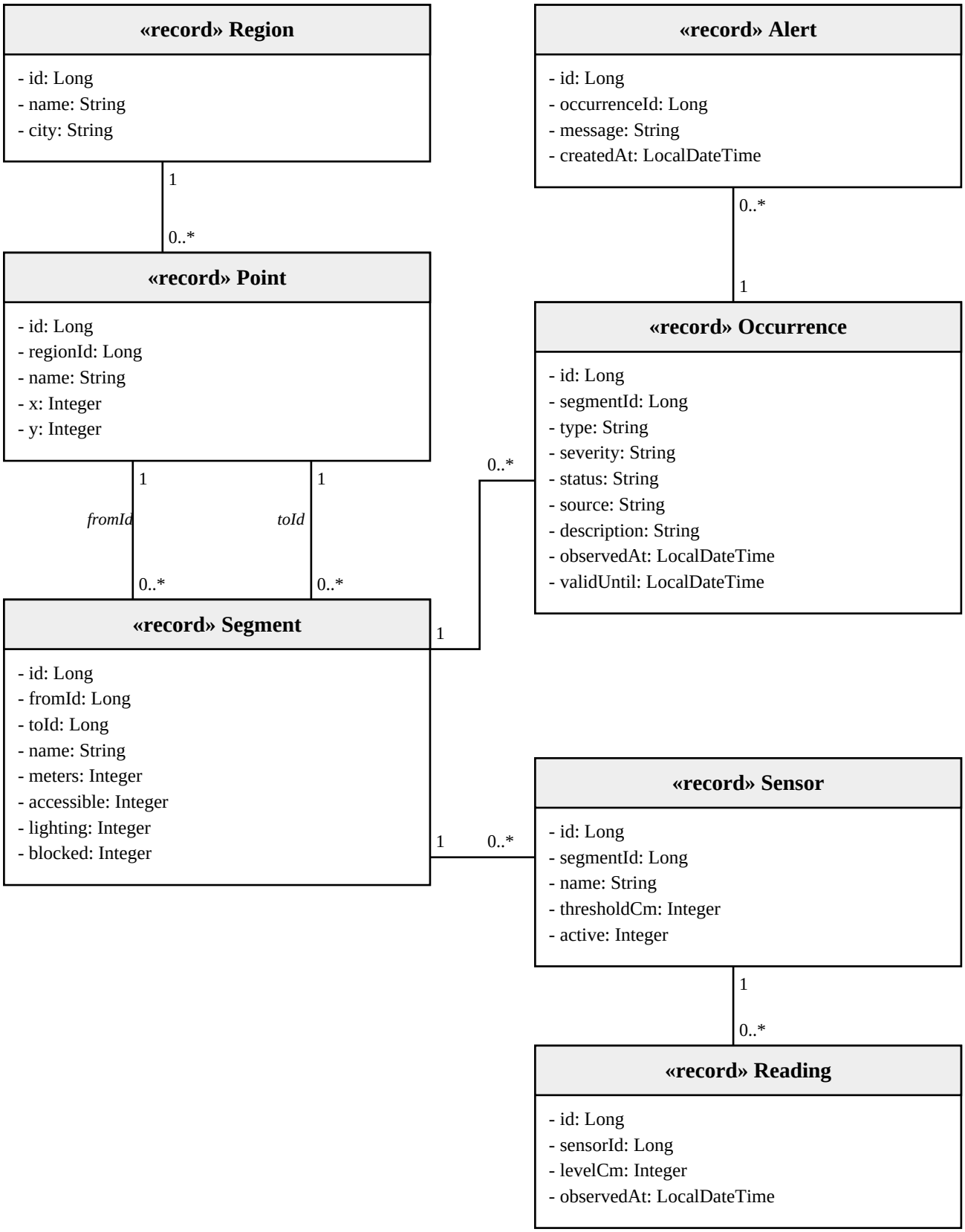


O fluxo de ocorrência usa três DAOs e RealNetwork. @Transactional mantém ocorrência e alertas na mesma operação. WeatherController delega a WeatherService, que usa HttpClient para acessar Open-Meteo.

Fluxo	Classes correspondentes
Cadastros	Region / Point / Segment / Sensor + Controller, BO e DAO de cada tipo.
Leituras e alertas	MonitoringController → ReadingBO, ReadingDAO, AlertDAO e DAOs do grafo.
Trajetos	RouteController → RouteBO → PointDAO, SegmentDAO, OccurrenceDAO e RealNetwork.
Malha real	RealNetwork: snapshot OSM + JdbcTemplate + TransactionTemplate; importa apenas registros ausentes.
Meteorologia	WeatherController → WeatherService → HttpClient / ObjectMapper; cache de 15 minutos.
Infraestrutura	ApiAccess: chave de operador. ApiErrors: tratamento centralizado. Missing e Conflict: exceções de domínio.

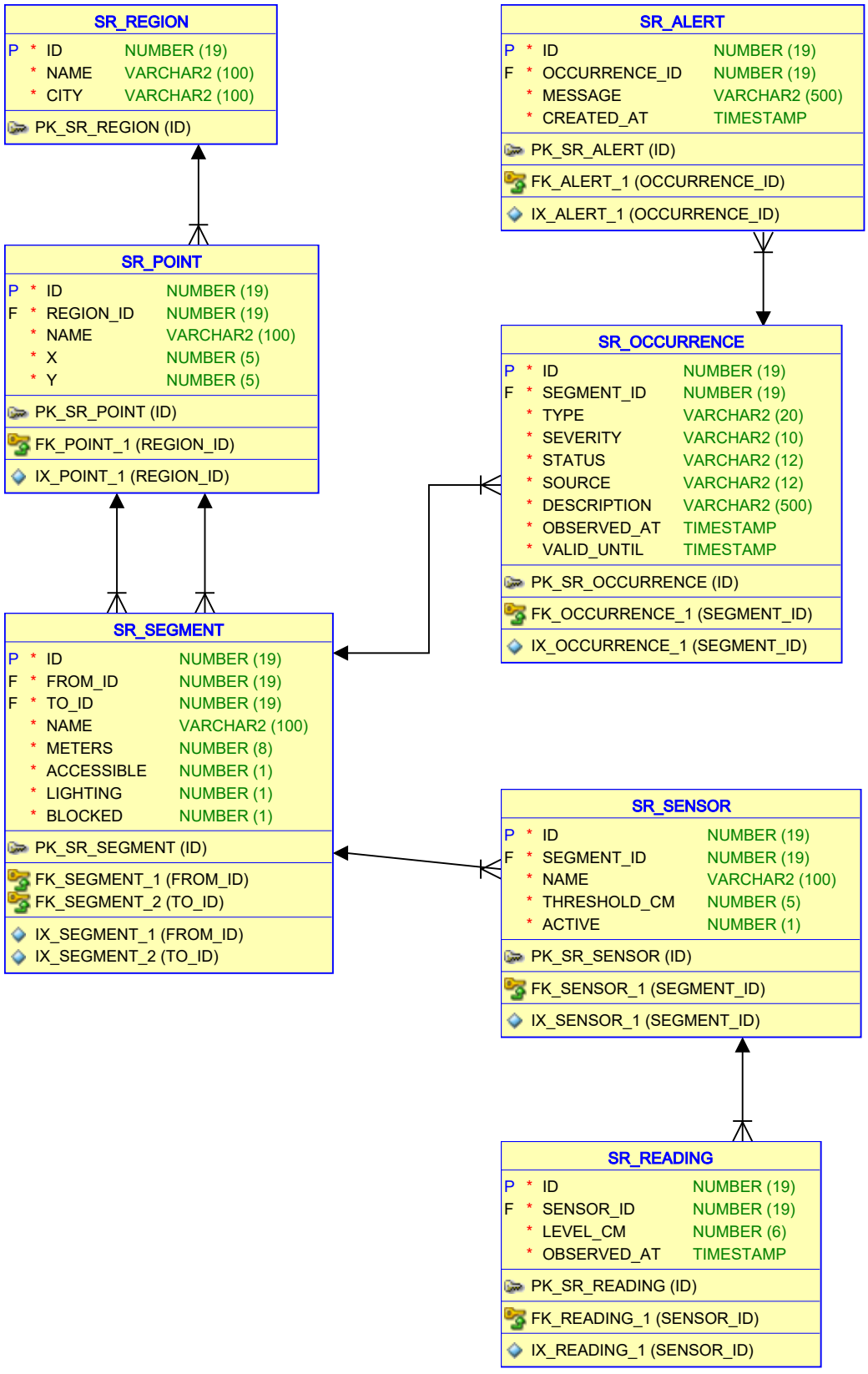
3. Diagrama de classes: domínio

Classes do pacote com.tlean07.saferoute, implementadas como records imutáveis. As associações abaixo são representadas por identificadores (Long) no código e por FKs no Oracle.



4. Modelo relacional

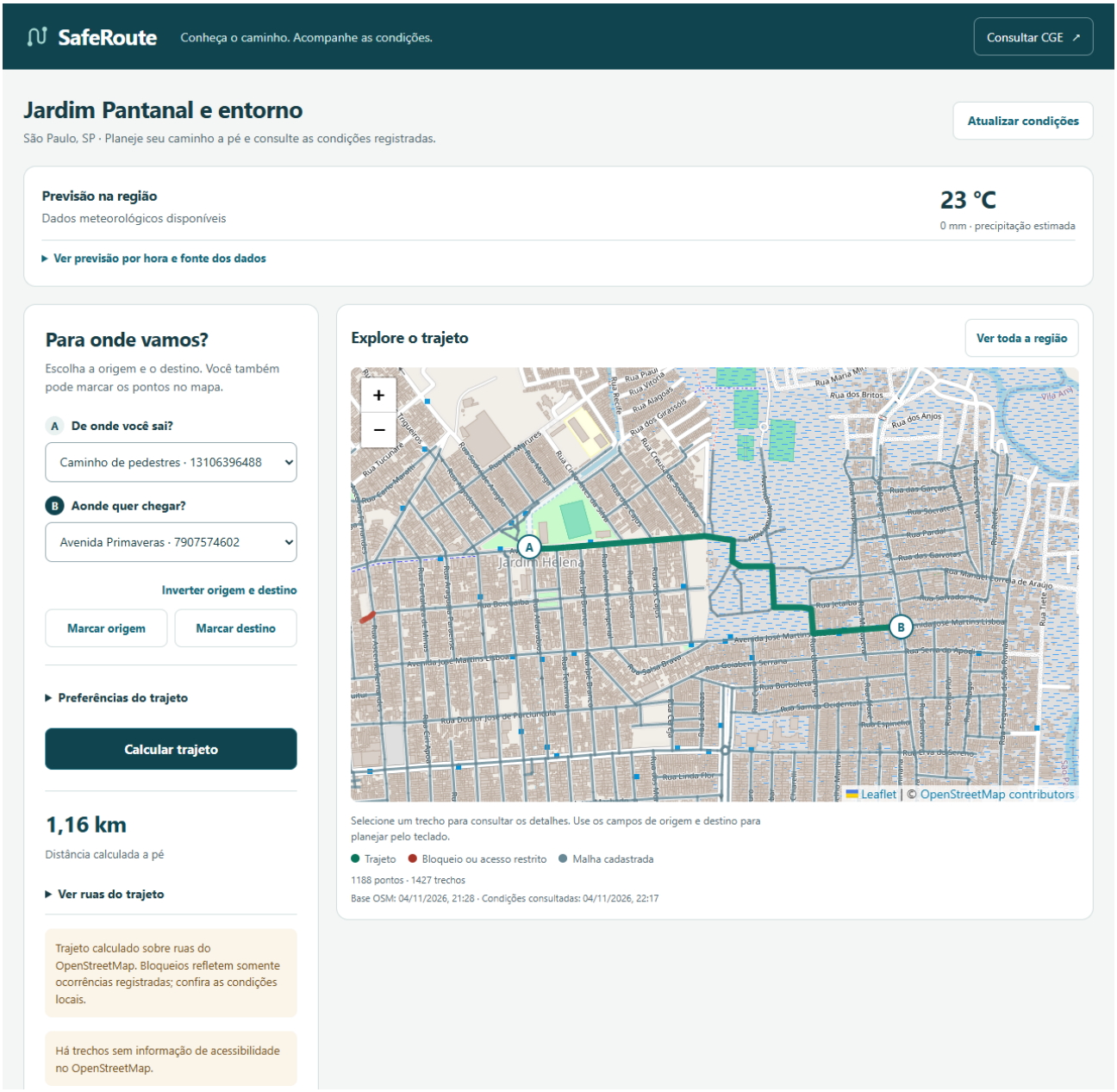
Modelo relacional completo, importado e exportado no Oracle SQL Developer Data Modeler 24.3.1. P = chave primária; F = chave estrangeira; * = atributo obrigatório.



Cada filho tem um pai obrigatório; cada pai pode ter 0..N filhos. Origem e destino são FKs distintas.

5. Interface: consulta de trajetos

Captura da interface publicada no Render, consumindo dados do Oracle FIAP. A origem e o destino são selecionados pelo usuário; a alternativa aparece no mapa e em texto.

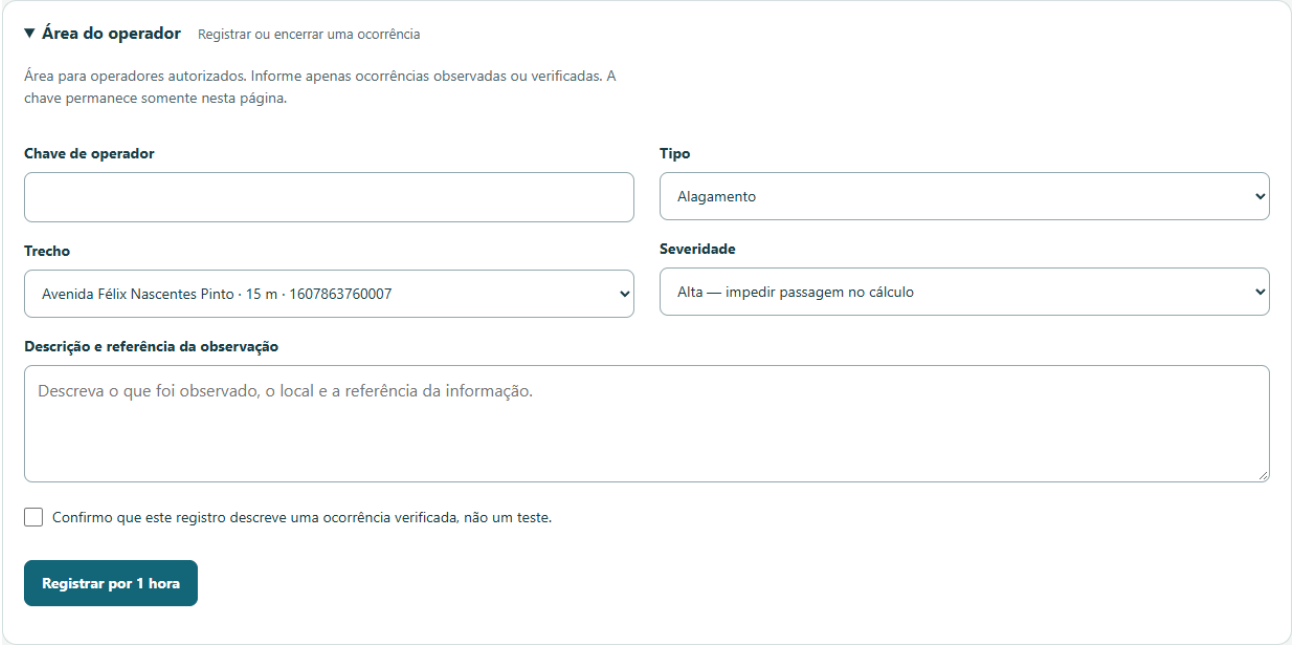


Cenário verificado: origem e destino na malha OSM, sem exigir acessibilidade não cadastrada. O trajeto padrão tem cerca de 1,16 km, passando por vias como Avenida Kumaki Aoki, Rua Ubapitanga e Avenida José Martins Lisboa.

O mapa usa coordenadas reais, tiles OpenStreetMap e atribuição visível. A previsão por hora e a fonte podem ser expandidas. A interface permite inverter os pontos, cancelar a seleção no mapa e reenquadrar a região. Ausência de ocorrência não é confirmação de passagem.

6. Interface: operação e ocorrências

O painel registra ocorrências verificadas pelo operador, com descrição, origem e validade de uma hora. As alterações exigem chave; a consulta de trajetos é pública. Registros simulados são recusados na malha real.



Ação na tela	Requisição e efeito
Carregar rede	GET /api/graph; lista pontos, trechos e ocorrências.
Consultar alternativa	POST /api/routes; retorna FOUND ou NO_ROUTE e avisos.
Registrar alagamento	POST /api/occurrences; persiste a ocorrência e o alerta.
Encerrar ocorrência	PUT /api/occurrences/{id} com status CLOSED; deixa de aparecer nos alertas ativos.
Meteorologia	GET /api/weather; retorna CURRENT, STALE ou UNAVAILABLE.
Atualizar dados	GET /api/graph e /api/weather; recalcule o trajeto após alterações.

7. Endpoints e requisições HTTP

Base pública: https://saferoute-gs-2026.onrender.com. O corpo das requisições é JSON. Cadastros usam GET, POST, PUT e DELETE conforme a tabela.

Recurso	Métodos e caminhos	Retorno
Regiões	GET/POST /api/regions GET/PUT/DELETE /api/regions/{id}	Region
Pontos	GET/POST /api/points GET/PUT/DELETE /api/points/{id}	Point
Trechos	GET/POST /api/segments GET/PUT/DELETE /api/segments/{id}	Segment
Sensores	GET/POST /api/sensors GET/PUT/DELETE /api/sensors/{id}	Sensor
Ocorrências	GET/POST /api/occurrences GET/PUT/DELETE /api/occurrences/{id}	Occurrence
Monitoramento	GET/POST /api/readings GET /api/alerts	Reading / alertas
Dados externos	GET /api/graph GET /api/weather GET /api/health	Grafo / previsão / UP
Trajetos	POST /api/routes	RouteBO.Result

Exemplo de consulta:

POST /api/routes
{"fromId":13106396488,"toId":7907574602,"accessible":false,"night":false}

Exemplo de cadastro de região:

POST /api/regions | X-Admin-Key: sua-chave
{"name":"Setor de teste","city":"São Paulo"}

GET e PUT retornam 200; POST de cadastro retorna 201 com Location; DELETE retorna 204. Erros: 400 entrada inválida; 401 chave ausente/incorreta; 404 registro inexistente; 409 integridade ou conflito; 503 banco ou meteorologia indisponível; 500 falha inesperada.

8. Como executar e acessar

O código completo está em **src/**, dentro do ZIP, junto de pom.xml, Maven Wrapper, SQL e testes. Extraia a pasta e siga os passos abaixo.
Referência exigida pelo enunciado: https://github.com/TLean07/saferoute-gs-2026
Deploy: https://saferoute-gs-2026.onrender.com

Ferramenta	Versão / finalidade
JDK / Spring Boot	Java 21 / Spring Boot 3.3.5.
Maven Wrapper	3.9.11; incluído no código-fonte.
Oracle / SQLcl	Oracle FIAP 19c / SQLcl 26.2.
Data Modeler	24.3.1; projeto nativo incluído no ZIP de Database.
Docker / Render	Build Maven + imagem Temurin 21; serviço gratuito.
Editor e verificação	IntelliJ IDEA 2025.3 ou editor compatível; JUnit, H2 e Playwright.

8.1 Demonstração local

1. Extraia o ZIP e abra a pasta do projeto com JDK 21 configurado.
2. No PowerShell, execute os comandos abaixo.
3. Abra http://localhost:8080. O perfil demo usa H2 e a mesma malha real OSM.

```
.\mvnw.cmd test
.\mvnw.cmd spring-boot:run "-Dspring-boot.run.profiles=demo"
```

Chave local do painel: **demo-saferoute-local**. O arquivo INICIAR_DEMO.cmd também inicia a demonstração. Linux/macOS: use sh mvnw.

8.2 Oracle e Render

Execute sql/01_ddl.sql e sql/02_dml.sql uma vez, em namespace SR_ vazio. Defina DB_URL, DB_USER, DB_PASSWORD e ADMIN_API_KEY no ambiente; use jdbc:oracle:thin:@//oracle.fiap.com.br:1521/ORCL. A chave deve ter pelo menos 16 caracteres. Inicie sem o perfil demo para usar Oracle. RealNetwork importa os registros ausentes do extrato OSM na região 900, sem alterar bloqueios existentes.

No Render, use Dockerfile, plano gratuito e health check /api/health. O deploy desta entrega foi verificado com o Oracle. Credenciais permanecem privadas; o avaliador pode testar alterações no perfil demo. O primeiro acesso após inatividade pode levar mais tempo.

9. Verificação e limites

Nível de verificação	Resultado observado em 04/11/2026
Testes locais	17 testes passaram: CRUD, rota real, isolamento da simulação, importação idempotente e cache/falhas meteorológicas.
API publicada + Oracle	Health UP; grafo real com 1.188 pontos/1.427 segmentos; trajeto calculado e Open-Meteo consultado. CRUD e autorização preservados.
Banco via MCP SQLcl	Sete tabelas, 2.655 registros após a importação real. Dados de teste ficam fora do mapa público.
Interface	Consulta de alternativa e painel de operador inspecionados no navegador. Capturas nas páginas 7 e 8.
Modelo Oracle	DDL importado sem erros; projeto salvo, reaberto e exportado no Data Modeler.

9.1 Restrições assumidas

Ruas e coordenadas provêm do OpenStreetMap; a meteorologia é uma estimativa de modelo regional, não medição por rua. Não há integração de alagamentos em tempo real, criminalidade ou sensores oficiais. O extrato atual não possui wheelchair=yes: exigir acessibilidade cadastrada retorna NO_ROUTE, sem presumir condições desconhecidas.

Os pesos de rota são heurísticos. Open-Meteo usa cache de 15 minutos e timeout de 15 segundos. Em falha, o último retorno de até 2 horas é marcado STALE; depois disso, retorna 503 sem inventar valores. A malha OSM é um extrato versionado, não uma atualização contínua. Registros e sensores de teste não afetam o mapa público.

9.2 Referências e rastreabilidade

Território: Prefeitura de São Paulo, Urbanismo Social Jardim Pantanal. Fonte oficial.

FIAP. GS CitizenTech 2026, orientações de Database Design e Domain Driven Design - Java, p. 31-33. Proposta SafeRoute fornecida pelo grupo. Render: documentação oficial de Docker e health checks.

Fontes: OpenStreetMap contributors (ODbL 1.0), Overpass API e Open-Meteo (CC BY 4.0). O JSON do extrato registra data, limites, IDs e coordenadas. Instruções e limites estão no README; testes em src/test/java/; exemplos em requisicoes.http.